

Android 第七章第一节教学讲义

(ContentProvider 与权限管理)

各位同学，今天咱们学习 Android 中实现跨应用数据共享的核心组件——**ContentProvider（内容提供者）**，以及非常重要的**运行时权限申请**。想象一下：你的 APP 需要读取手机通讯录，或者另一款 APP 需要使用你的 APP 存储的数据——这些跨应用数据交互，都离不开 ContentProvider。同时，为了用户隐私安全，Android 要求危险权限必须在运行时动态申请。这节课咱们用 "原理 + 代码 + 案例" 的方式，保证小白也能轻松掌握！

一、ContentProvider：跨应用数据共享的 "桥梁"

1. 什么是 ContentProvider?

简单说，它是 Android 四大组件之一，专门用于**不同 APP 之间共享数据**。比如手机的通讯录 APP 通过 ContentProvider 暴露数据，其他 APP（如微信）就可以通过它读取联系人信息。

就像：

- 通讯录 APP（数据拥有者）→ 用 ContentProvider 搭建一座 "桥"；
- 微信（数据使用者）→ 通过这座 "桥" 读取联系人。

2. 核心概念：URI（数据的 "地址"）

要访问 ContentProvider 中的数据，必须知道它的 "地址"——**URI（统一资源标识符）**，格式如下：

`content://authority/path/id`

- `content://`：固定前缀，标识这是 ContentProvider 的 URI；
- `authority`：唯一标识（通常是应用包名，如 `com.example.provider`），防止不同 ContentProvider 冲突；
- `path`：数据路径（通常是表名，如 `users`），标识要访问的数据；
- `id`：可选，指定某条数据的 ID（如 `1`）。

例子：

`content://com.example.provider/users` → 访问`com.example.provider`下的`users`表所有数据；

`content://com.example.provider/users/1` → 访问`users`表中 ID 为 1 的数据。

二、Server 端：实现 ContentProvider（数据提供方）

假设我们有一个 APP（Server 端），用 SQLite 存储了用户数据，现在要通过 ContentProvider 让其他 APP 访问这些数据。

步骤 1：准备数据（SQLite 数据库）

先创建一个存储用户数据的数据库（复用之前的 SQLiteOpenHelper）：

```
// 数据库帮助类
public class UserDbHelper extends SQLiteOpenHelper {
    public static final String DB_NAME = "user_db";
    public static final int VERSION = 1;
    public UserDbHelper(Context context) {
        super(context, DB_NAME, null, VERSION);
    }
    @Override
    public void onCreate(SQLiteDatabase db) {
        // 创建用户表
        db.execSQL("CREATE TABLE user (" +
            "id INTEGER PRIMARY KEY AUTOINCREMENT," +
            "name TEXT NOT NULL," +
            "age INTEGER)");
    }
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {}
}
```

步骤 2：定义 ContentProvider

创建一个类继承`ContentProvider`，重写 6 个核心方法，实现数据的增删改查逻辑。

```
public class UserProvider extends ContentProvider {
    // 1. 定义 authority（必须和Manifest中注册的一致）
    public static final String AUTHORITY = "com.example.userprovider";
    // 2. 定义URI匹配器（用于匹配不同的URI）
    private static final UriMatcher URI_MATCHER = new UriMatcher(UriMatcher.NO_MATCH);
    // 3. 定义匹配码（区分不同的URI）
    private static final int USERS = 1; // 匹配users表所有数据
    private static final int USER_ID = 2; // 匹配users表单条数据
    // 静态代码块：初始化URI匹配器
    static {
        // 匹配 content://com.example.userprovider/users
        URI_MATCHER.addURI(AUTHORITY, "users", USERS);
        // 匹配 content://com.example.userprovider/users/1
        URI_MATCHER.addURI(AUTHORITY, "users/#", USER_ID);
    }
    private UserDbHelper dbHelper;
    private SQLiteDatabase db;
    @Override
    public boolean onCreate() {
        // 初始化数据库
        dbHelper = new UserDbHelper(getContext());
        db = dbHelper.getWritableDatabase();
        return true;
    }
    // 4. 查询数据（核心方法）
    @Nullable
    @Override
    public Cursor query(
        @NonNull Uri uri,
        @Nullable String[] projection,
        @Nullable String selection,
        @Nullable String[] selectionArgs,
        @Nullable String sortOrder) {
        Cursor cursor = null;
        // 根据URI匹配码执行不同查询
        switch (URI_MATCHER.match(uri)) {
            case USERS:
                // 查询所有用户
                cursor = db.query("user", projection, selection, selectionArgs, null, null, sortOrder);
                break;
            case USER_ID:
                // 查询单个用户
                cursor = db.query("user", projection, selection + " and _id = ?", selectionArgs, null, null, sortOrder);
                break;
            default:
                return null;
        }
        return cursor;
    }
}
```

```

        // 查询单条用户（从URI中获取ID）
        String id = uri.getPathSegments().get(1); // 第1个路径片段是ID
        cursor = db.query("user", projection, "id=?", new String[]{id}, null, null, sortOrder);
        break;
    default:
        throw new IllegalArgumentException("未知URI: " + uri);
    }
    return cursor;
}

// 5. 其他方法（插入、更新、删除、获取MIME类型）
@Nullable
@Override
public String getType(@NonNull Uri uri) {
    // 简单返回MIME类型（根据需求实现）
    return null;
}

@Nullable
@Override
public Uri insert(@NonNull Uri uri, @Nullable ContentValues values) {
    // 插入数据逻辑（类似查询，根据匹配码处理）
    return null;
}

@Override
public int delete(@NonNull Uri uri, @Nullable String selection, @Nullable String[]
selectionArgs) {
    // 删除数据逻辑（案例中会详细实现）
    return 0;
}

@Override
public int update(@NonNull Uri uri, @Nullable ContentValues values, @Nullable String
selection, @Nullable String[] selectionArgs) {
    // 更新数据逻辑
    return 0;
}
}

```

步骤 3：在 Manifest 中注册 ContentProvider

必须在 Server 端的 `AndroidManifest.xml` 中注册，声明 `authority` 和权限：

```
<manifest ...>
  <application ...>
    <!-- 注册ContentProvider -->
    <provider
      android:name=".UserProvider"
      android:authorities="com.example.userprovider" <!-- 和代码中AUTHORITY一致 -->
      android:exported="true" <!-- 允许其他APP访问 -->
      android:permission="com.example.READ_USER" /> <!-- 访问需要的权限（可选） -->
    </provider>
  </application>
</manifest>
```

- `exported="true"`：允许外部 APP 访问；
- `permission`：可选，限制只有声明了该权限的 APP 才能访问。

三、Client 端：访问 ContentProvider（数据使用方）

另一款 APP（Client 端）通过 `ContentResolver` 访问 Server 端的 `ContentProvider`。

核心工具：ContentResolver

Android 提供 `ContentResolver` 类，统一处理跨应用数据访问，方法和 `ContentProvider` 对应（`query/insert/update/delete`）。

步骤 1：查询 Server 端数据

```
// Client端Activity中
public class ClientActivity extends AppCompatActivity {
    private TextView tvResult;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_client);
        tvResult = findViewById(R.id.tv_result);
        // 点击按钮查询用户数据
        findViewById(R.id.btn_query).setOnClickListener(v -> queryUsers());
    }
    private void queryUsers() {
```

```

// 1. 获取ContentResolver
ContentResolver resolver = getContentResolver();
// 2. 定义要访问的URI (Server端的authority + 路径)
Uri uri = Uri.parse("content://com.example.userprovider/users");
// 3. 查询数据 (参数和SQLite的query类似)
Cursor cursor = resolver.query(
    uri,
    new String[]{"id", "name", "age"}, // 要查询的列
    null, // 条件 (如"age>18")
    null, // 条件参数
    "id DESC" // 排序
);
// 4. 处理查询结果
if (cursor != null && cursor.moveToFirst()) {
    StringBuilder sb = new StringBuilder();
    do {
        int id = cursor.getInt(cursor.getColumnIndex("id"));
        String name = cursor.getString(cursor.getColumnIndex("name"));
        int age = cursor.getInt(cursor.getColumnIndex("age"));
        sb.append("ID:").append(id).append(", 姓名:").append(name).append(", 年龄:")
.append(age).append("\n");
    } while (cursor.moveToNext());
    tvResult.setText(sb.toString());
    cursor.close(); // 关闭游标
} else {
    tvResult.setText("没有数据");
}
}
}

```

步骤 2：案例：删除 Server 端数据

在 Client 端实现删除 Server 端指定 ID 的用户：

```

// Client端删除按钮点击事件
findViewById(R.id.btn_delete).setOnClickListener(v -> {
    ContentResolver resolver = getContentResolver();
    // 要删除的用户ID为1，URI格式：content://authority/users/1
    Uri uri = Uri.parse("content://com.example.userprovider/users/1");

```

```

// 执行删除
int rowsDeleted = resolver.delete(uri, null, null);
if (rowsDeleted > 0) {
    Toast.makeText(this, "删除成功", Toast.LENGTH_SHORT).show();
    queryUsers(); // 重新查询刷新列表
} else {
    Toast.makeText(this, "删除失败", Toast.LENGTH_SHORT).show();
}
});
// 对应的Server端UserProvider中完善delete方法：
@Override
public int delete(@NonNull Uri uri, @Nullable String selection, @Nullable String[]
selectionArgs) {
    int rowsDeleted = 0;
    switch (URI_MATCHER.match(uri)) {
        case USERS:
            // 删除所有符合条件的用户
            rowsDeleted = db.delete("user", selection, selectionArgs);
            break;
        case USER_ID:
            // 删除指定ID的用户
            String id = uri.getPathSegments().get(1);
            rowsDeleted = db.delete("user", "id=?", new String[]{id});
            break;
        default:
            throw new IllegalArgumentException("未知URI: " + uri);
    }
    return rowsDeleted;
}
}

```

四、运行时动态申请权限：保护用户隐私

Android 6.0（API 23）以上，**危险权限**（如读取通讯录、存储、相机等）必须在 APP 运行时向用户申请，用户同意后才能使用。

1. 危险权限 vs 普通权限

- **普通权限**：不涉及隐私（如网络访问），在 Manifest 中声明即可；

- **危险权限**：涉及隐私（如读取联系人、拍照、定位），必须运行时申请。

2. 动态申请权限的两种方式

方式 1：Lazy（懒加载式）—— 用到时再申请

比如点击 "读取联系人" 按钮时才申请权限：

```
// 权限常量（读取联系人权限）
private static final String READ_CONTACTS_PERMISSION = Manifest.permission.READ_CONTACTS;
private static final int PERMISSION_REQUEST_CODE = 100;
// 点击按钮读取联系人（需要权限）
findViewById(R.id.btn_read_contacts).setOnClickListener(v -> {
    // 1. 检查是否已有权限
    if (ContextCompat.checkSelfPermission(this, READ_CONTACTS_PERMISSION)
        == PackageManager.PERMISSION_GRANTED) {
        // 已有权限，直接执行操作
        readContacts();
    } else {
        // 2. 没有权限，申请权限
        ActivityCompat.requestPermissions(
            this,
            new String[]{READ_CONTACTS_PERMISSION}, // 要申请的权限数组
            PERMISSION_REQUEST_CODE // 请求码（自定义，用于回调中判断）
        );
    }
});
// 3. 权限申请结果回调
@Override
public void onRequestPermissionsResult(
    int requestCode,
    @NonNull String[] permissions,
    @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == PERMISSION_REQUEST_CODE) {
        // 判断用户是否同意
        if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED
    ) {
```



```

        // 用户同意，执行操作
        readContacts();
    } else {
        // 用户拒绝，提示无法使用功能
        Toast.makeText(this, "需要权限才能读取联系人", Toast.LENGTH_SHORT).show();
    }
}
}
// 实际读取联系人的方法（略）
private void readContacts() { ... }

```

方式 2: Hungry（饥饿式）—— APP 启动时申请

在 APP 启动（如 Splash 页）时就申请必要的权限：

```

public class SplashActivity extends AppCompatActivity {
    private static final String[] REQUIRED_PERMISSIONS = {
        Manifest.permission.READ_EXTERNAL_STORAGE,
        Manifest.permission.CAMERA
    }; // 需要的权限数组
    private static final int PERMISSION_REQUEST_CODE = 200;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_splash);
        // 启动时检查并申请权限
        checkAndRequestPermissions();
    }
    private void checkAndRequestPermissions() {
        // 收集未授权的权限
        List<String> needPermissions = new ArrayList<>();
        for (String permission : REQUIRED_PERMISSIONS) {
            if (ContextCompat.checkSelfPermission(this, permission)
                != PackageManager.PERMISSION_GRANTED) {
                needPermissions.add(permission);
            }
        }
        if (needPermissions.isEmpty()) {
            // 所有权限都已授权，进入首页
            startMainActivity();
        }
    }
}

```

```

    } else {
        // 申请未授权的权限
        ActivityCompat.requestPermissions(
            this,
            needPermissions.toArray(new String[0]),
            PERMISSION_REQUEST_CODE
        );
    }
}

@Override
public void onRequestPermissionsResult(...) {
    // 检查是否所有权限都被同意
    boolean allGranted = true;
    for (int result : grantResults) {
        if (result != PackageManager.PERMISSION_GRANTED) {
            allGranted = false;
            break;
        }
    }
    if (allGranted) {
        startMainActivity(); // 进入首页
    } else {
        Toast.makeText(this, "部分权限被拒绝，功能可能受限", Toast.LENGTH_SHORT).show();
        startMainActivity(); // 也可以强制退出APP
    }
}

private void startMainActivity() {
    startActivity(new Intent(this, MainActivity.class));
    finish();
}
}

```

注意：权限申请的 "潜规则"

- 向用户说明为什么需要权限（如 "需要相机权限用于拍照"）；
- 如果用户拒绝并勾选 "不再询问"，需要引导用户去设置页手动开启权限；
- 分组权限：同一组的权限（如READ_EXTERNAL_STORAGE和WRITE_EXTERNAL_STORAGE同属存储组），申请一个即可获得整组权限。

总结

这节课咱们学了两个核心知识点：

1. **ContentProvider:**

- 用于跨 APP 数据共享，通过 URI 标识数据地址；
- Server 端：继承 `ContentProvider` 实现增删改查，在 Manifest 中注册；
- Client 端：通过 `ContentResolver` 访问 Server 端的 URI，实现数据交互。

2. **运行时动态权限:**

- 危险权限必须在运行时申请，分 Lazy（用到时申请）和 Hungry（启动时申请）两种方式；
- 通过 `checkSelfPermission` 检查权限，`requestPermissions` 申请权限，`onRequestPermissionsResult` 处理结果。

课后作业：用 ContentProvider 实现一个 "笔记 APP"，允许其他 APP 读取和添加笔记，并在访问时动态申请 "读写笔记" 权限（自定义权限）。练完就能熟练掌握这些知识点啦！有问题随时问～

（注：文档部分内容可能由 AI 生成）